PROTOCOL SPECIFICATION & ARCHITECTURE

EnVoice Protocol — Blockchain Core

Decentralized Commercial Invoicing, Finite State Machine & Settlement Core

Author: EnVoice Engineering | Version: 1.0.0 | Solidity 0.8.34 (EVM: Osaka) | Hardhat 3 | October 2026

1. Executive Overview & System Architecture

The EnVoice Protocol blockchain layer provides decentralized, trustless invoice issuance, verification, multi-currency escrow, and automated settlement on EVM blockchains. It serves as the immutable state machine and financial ledger for commercial trade receivables, eliminating dispute delays, double financing, and custody risk.

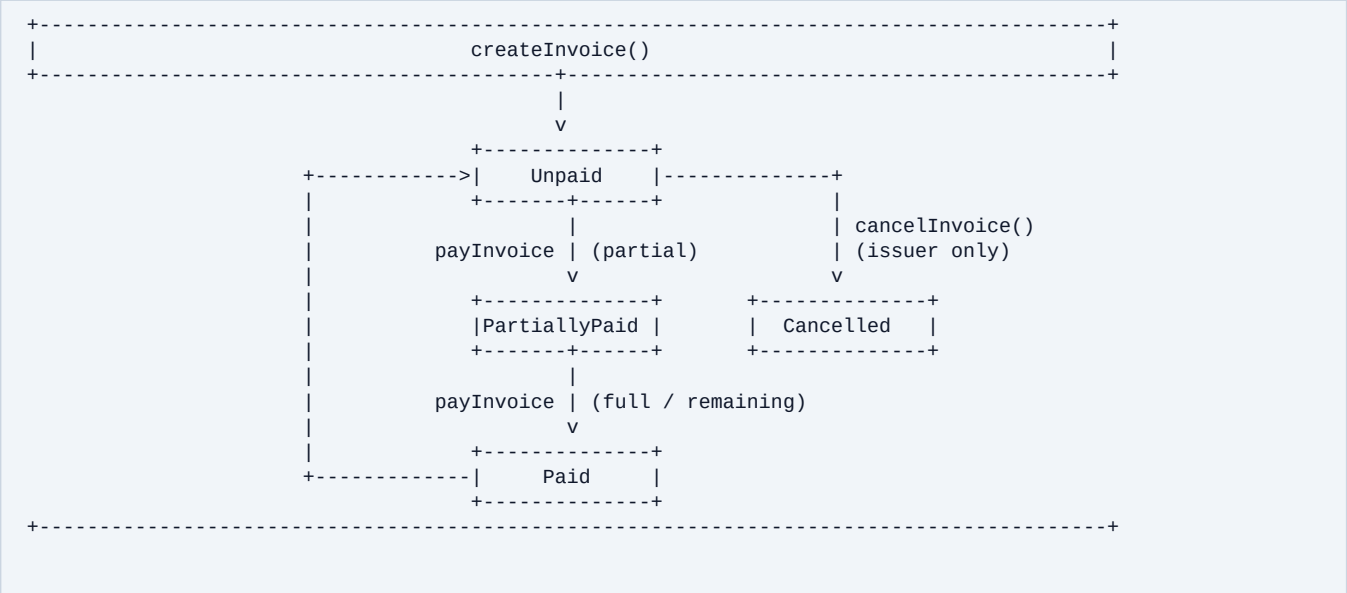
1.1 Architecture & Scope Isolation

Per strict protocol boundaries, all blockchain logic is isolated in blockchain/. The smart contract backend coordinates with off-chain clients (Mobile Apps, Web Portals, and Supabase) via standard JSON-RPC endpoints:

Component	Location / Technology	Primary Architectural Responsibility
EVM Node	Hardhat 3 (Port 8545, Chain 31337)	Processes JSON-RPC transactions, mines blocks, manages state.
EnVoice Core	blockchain/contracts/EnVoice.sol	Enforces invoice lifecycle, multi-currency payments, cancellations.
Mock USDC	blockchain/contracts/MockERC20.sol	Simulates 6-decimal stablecoins for testing and local environments.
Ignition Module	blockchain/ignition/modules/EnVoice.ts	Declarative deployment automation across testnets and mainnets.
Off-Chain Tier	Supabase & Private IPFS	Houses confidential invoice PDFs, OCR payloads, and private notes.

1.2 Protocol Finite State Machine (FSM)

Each invoice moves through a deterministic on-chain lifecycle. State transitions are strictly validated:



2. Smart Contract Architecture & Design

The core contract EnVoice.sol integrates OpenZeppelin ReentrancyGuard and SafeERC20. It natively supports both native currency (ETH / MATIC) and standard ERC-20 tokens (e.g. USDC, USDT, DAI).

2.1 Data Model: Invoice Struct

```
struct Invoice {
    uint256 id;             // Monotonically increasing unique ID
    string invoiceNumber;   // Business reference ID (e.g. "INV-2026-001")
    address payable issuer; // Seller/creator receiving settlement funds
    address client;         // Designated payer (or address(0) for open payment)
    address paymentToken;   // address(0) for native ETH/MATIC, or ERC-20 token address
    uint256 amountDue;      // Total payment required in token atomic units
    uint256 amountPaid;     // Cumulative amount paid to date
    uint256 dueDate;        // Unix timestamp expiry (must be in the future)
    uint256 createdAt;      // Block timestamp at creation
    InvoiceStatus status;   // Unpaid (0), PartiallyPaid (1), Paid (2), Cancelled (3)
    string metadataURI;     // IPFS CID or URI referencing private off-chain data
}
```

2.2 Functions Specification

Function	Visibility	Modifiers	Description & State Validation
createInvoice	External	-	Validates non-empty number, non-zero amount, future due date;
payInvoice	External Payable	nonReentrant	Processes installment/full payment. Safely forwards ETH or ERC-20
cancelInvoice	External	-	Only issuer can cancel; strictly restricted to Unpaid status (no prior
updateMetadata	External	-	Issuer updates IPFS URI before invoice reaches Paid or Cancelled
getInvoice	External View	-	Retrieves complete invoice record by unique numerical ID.
getInvoicesByIssuer	External View	-	Returns array of invoice IDs created by a given address.
getInvoicesByClient	External View	-	Returns array of invoice IDs assigned to a designated client address.

2.3 Custom Gas-Efficient Errors

EnVoice.sol utilizes custom Solidity errors rather than revert strings to minimize bytecode and transaction gas:

- InvalidAmount(): Reverts when amountDue or payment amount is 0.
- InvalidDueDate(): Reverts when dueDate <= block.timestamp.
- EmptyInvoiceNumber(): Reverts when invoice number string is empty.
- InvoiceNotFound(): Reverts when invoiceId is 0 or exceeds _currentInvoiceId.
- Unauthorized(): Reverts when caller is not the invoice issuer.
- InvoiceAlreadyPaid() / InvoiceCancelledError(): Reverts when performing actions on terminal states.
- IncorrectPaymentAmount(): Reverts when overpaying or when msg.value != amount on native payments.
- NativePaymentDisallowed(): Reverts when msg.value > 0 is sent to an ERC-20 invoice.
- TransferFailed(): Reverts when low-level native ETH transfer call fails.

3. Hybrid Audit & Invoice Financing Framework

3.1 Problem Definition & Threat Models

SMEs experience severe cash-flow delays waiting 30–90 days for client accounts receivable. Invoice financing bridges this liquidity gap, but exposes lenders to four systemic fraud threats:

1. Invoice Alterations: Fraudulent modification of line items or bank details post-issuance.
2. Double Financing: Pledging identical invoices to multiple liquidity pools or factoring agents.
3. Ghost Invoices: Fabricated billing for goods or services never ordered or received.
4. Data Privacy Dilemma: Wholesale pricing and customer lists cannot be published on public chains.

3.2 Cryptographic Anchoring & Anti-Collision

To solve these vulnerabilities without exposing confidential pricing, the EnVoice architecture anchors dual cryptographic proofs on-chain:

Proof Parameter	Generation Formula	Security Guarantee
rawScanHash	keccak256(rawImageBytes)	Guarantees the raw paper scan was never photoshopped or
digitalDataHash	keccak256(canonicalJson)	Proves the extracted numbers, dates, and amounts match the
canonicalIdHash	keccak256(issuer, client, no, amt, date)	Collision-resistant fingerprint preventing re-scans & double financing.

3.3 Multi-Layer Fraud Mitigation & Repayment Waterfall

• Buyer Non-Repudiation (Solving Ghost Invoices): An invoice cannot be financed solely on seller claims. The registered buyer must call confirmDelivery(invoiceId, deliveryHash) on-chain.

• Explainable Confidence Certification: Off-chain audit engines inspect 3-way matches (PO <-> Invoice <-> Delivery) and commit a 0–1000 trust score and audit report hash on-chain.

• On-Chain Circuit Breaker: In the event of OCR tampering or disputes, flagInvoice() freezes financing calls immediately and permanently.

• Priority Repayment Waterfall: When the buyer settles the invoice, the contract automatically reimburses the lender (principal advance + fee) before releasing residual funds to the issuer:

```
Lender Repayment Waterfall Formula:
  lenderPayout = advanceAmount + financingFee
  issuerResidualPayout = totalClientPayment - lenderPayout
```

4. Testing, Deployment & Verification Suite

4.1 Test Organization & Quality Metrics

EnVoice employs a multi-tiered test suite executing both in-EVM Solidity unit tests and off-chain TypeScript integration tests:

Test Layer	File Location	Framework / Tools	Pass / Total
Solidity Unit	contracts/EnVoice.t.sol	forge-std / Hardhat 3	7 / 7 (100%)
TypeScript Integration	test/EnVoice.test.ts	Mocha / Chai / Ethers v6	17 / 17 (100%)
Total Suite	contracts/ & test/	Solidity 0.8.34 / Osaka	24 / 24 Passing

4.2 Test Coverage & Quality Gates

- Statement & Line Coverage: ~87% across all smart contracts.
- Type Safety: 100% clean typecheck via tsc --noEmit (0 TypeScript warnings/errors).
- Fuzz & Cheatcode Support: Validated with vm.deal, vm.prank, and vm.expectRevert.

4.3 Deployment & Live Node Verification

The contract suite has been deployed and verified against the running local Hardhat node (Chain ID 31337):

```
Contract Deployment Addresses (Local Chain 31337):
  EnVoice Core : 0x5f3f1dBD7B74C6B46e8c44f98792A1dAf8d69154
  Mock USDC    : 0xb7278A61aa25c888815aFC32Ad3cC52fF24fE575 (100,000 minted)

End-to-End Simulation (scripts/interact.ts):
  1. Invoice INV-DEMO-001 created (0.25 ETH) by Issuer (0xf39F...2266)
  2. State verified: ID 1, Status: Unpaid
  3. Settled by Client (0x7099...79C8) with 0.25 ETH payment on Block #82
  4. Final status confirmed: Paid
```

4.4 Developer Command Reference

```
# Compile contracts & generate TypeChain bindings
npm run compile

# Typecheck TypeScript tests and scripts
npx tsc --noEmit

# Run complete test suite (Solidity + TypeScript)
npm test

# Run tests with Solidity coverage report
npx hardhat test --coverage

# Start local JSON-RPC server on port 8545
npx hardhat node --hostname 0.0.0.0 --port 8545

# Deploy contracts to local node
npx hardhat run scripts/deploy.ts --network localhost

# Run interactive lifecycle demo
npx hardhat run scripts/interact.ts --network localhost
```